MECHATRONICS AND IOT

ASSIGNMENT REPORT – 1

TITLE:

Design and simulate an Air Pollution Monitoring System using MQ2 Gas Sensor. The system detects harmful gases and monitors air quality levels for industrial and environmental safety applications.

TEAM MEMBERS:

HARISANKARAN S
(24MER013)
MUTHUKUMAR M
(24MER034)
MONISH C
(24MER032)

PROJECT OVERVIEW

Heating, Ventilation and Air Conditioning (HVAC) systems are widely used in industries, commercial buildings, hospitals, offices, laboratories, and residential buildings. Continuous monitoring of HVAC parameters is important for maintaining indoor comfort, reducing energy consumption, and preventing equipment failures.

The proposed MQTT-Based Remote HVAC Monitoring System continuously monitors room temperature, relative humidity, supply-air temperature, return-air temperature, air quality, fan status, and compressor status. An ESP32 microcontroller collects sensor data and transmits the measurements through Wi-Fi using MQTT to the ThingSpeak Cloud platform.

ThingSpeak stores the sensor data and provides graphs for real-time monitoring and historical analysis. If temperature, humidity, or air quality exceeds predefined limits, the system generates a warning and can activate an alarm.

PROBLEM STATEMENT

Conventional HVAC systems generally operate using local thermostats and provide limited remote monitoring capabilities.

- Excessive room temperature.
- High or low humidity.
- Poor indoor air quality.
- HVAC equipment operating continuously.
- Increased energy consumption.
- Lack of remote monitoring.
- Delayed detection of abnormal conditions.
- Difficulty in maintaining historical HVAC data.

Therefore, an IoT-based HVAC monitoring system is required to continuously monitor environmental conditions and provide remote access to HVAC data.

PROPOSED SOLUTION

The proposed system uses an ESP32 controller, environmental sensors, and Wi-Fi connectivity. The ESP32 collects HVAC parameters and processes them according to predefined threshold values.

- Measures temperature and humidity.
- Measures supply and return air temperatures.
- Measures air-quality conditions.
- Determines HVAC operating status.
- Displays readings locally.
- Publishes sensor data using MQTT.
- Sends data to ThingSpeak Cloud.
- Stores historical readings for analysis.
- Activates an alert when abnormal conditions occur.

WORKING PRINCIPLE

1. Initialize the ESP32.
2. Initialize the temperature, humidity and air-quality sensors.
3. Connect the ESP32 to Wi-Fi.
4. Connect to the MQTT broker.
5. Read room temperature and humidity.
6. Read supply-air and return-air temperatures.
7. Read air-quality value.
8. Monitor fan and compressor status.
9. Compare readings with predefined HVAC limits.
10. Determine the HVAC condition as NORMAL, WARNING, or CRITICAL.
11. Display readings on the LCD.
12. Publish sensor data using MQTT.
13. Store the data in ThingSpeak Cloud.
14. Activate the buzzer when a critical condition is detected.
15. Repeat the process continuously.

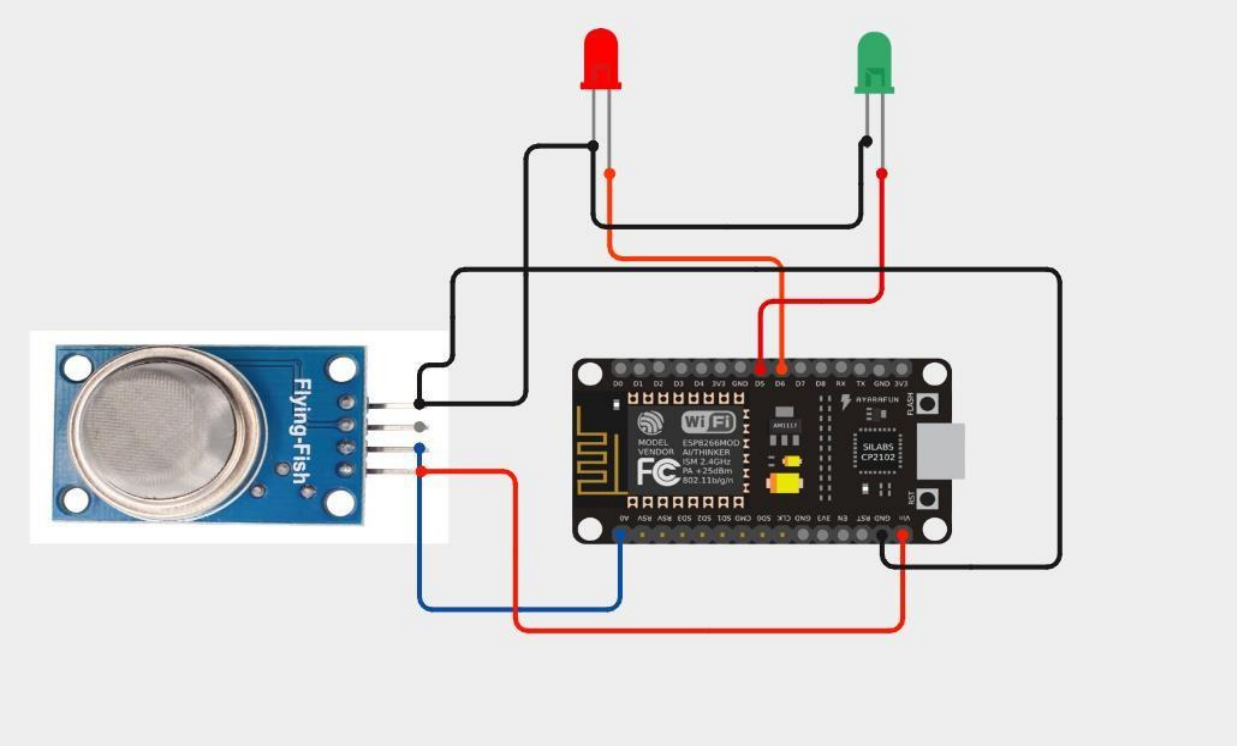
SYSTEM ARCHITECTURE

HVAC Sensors → ESP32 Microcontroller → Wi-Fi Network → MQTT Communication → ThingSpeak Cloud → Cloud Storage & Graphs → Remote Mobile/Web Dashboard

CIRCUIT TABLE

Component	**Pin / Interface**	**ESP32 Connection**	**Purpose**
DHT22	DATA	GPIO 4	Room temperature & humidity
DS18B20 – Supply Air	DATA	GPIO 5	Supply-air temperature
DS18B20 – Return Air	DATA	GPIO 18	Return-air temperature
MQ-135	Analog OUT	GPIO 34	Air-quality monitoring
Relay – Fan	IN	GPIO 26	Fan control/status
Relay – Compressor	IN	GPIO 27	Compressor control/status
Buzzer	Positive	GPIO 25	Warning alarm
16×2 I2C LCD	SDA/SCL	GPIO 21/22	Display HVAC values

CIRCUIT DESIGN



ALGORITHM

START
↓
Initialize ESP32
↓
Initialize HVAC Sensors
↓
Initialize LCD and Buzzer
↓
Connect to Wi-Fi
↓
Connect to MQTT Broker
↓
Read HVAC Parameters
↓
Compare with Threshold Values
↓
Any Abnormal Condition?
NO → NORMAL → Buzzer OFF
YES → WARNING/CRITICAL → Buzzer ON
↓
Display Data on LCD

↓

Publish Data using MQTT

↓

Upload/Store Data on ThingSpeak

↓

Repeat Continuously

CODING

```
#include <ESP8266WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// ================================
// WIFI
// ================================

#define WIFI_SSID       "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD   "qkts5737"

// ================================
// ADAFRUIT IO
// ================================

#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME    "Hair_13"
#define AIO_KEY         "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// ================================
// PINS
// ================================

#define MQ135_PIN A0
```

```
#define GREEN_LED D5
#define RED_LED D6

// ================================
// MQTT
// ================================

WiFiClient client;

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// ================================
// FEEDS
// ================================

Adafruit_MQTT_Publish mq135Feed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/mq135"
  );

Adafruit_MQTT_Publish airQualityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/air-quality"
  );

// ================================
```

```
// SETUP
// ================================

void setup()
{
  Serial.begin(9600);
  delay(1000);

  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);

  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  Serial.println();
  Serial.println("================================");
  Serial.println(" AIR POLLUTION MONITORING");
  Serial.println("================================");

  // --------------------
  // CONNECT WIFI
  // --------------------

  Serial.print("Connecting to WiFi");

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
```

```
  Serial.println("WiFi CONNECTED!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  // ---------------------
  // SENSOR WARM UP
  // ---------------------

  Serial.println();
  Serial.println("MQ135 sensor warming up...");
  delay(10000);

  Serial.println("Sensor ready!");
}

// ================================
// MQTT CONNECTION
// ================================

void MQTT_connect()
{
  if (mqtt.connected())
  {
    return;
  }

  Serial.println();
  Serial.print("Connecting to Adafruit IO...");

  int8_t ret;

  while ((ret = mqtt.connect()) != 0)
  {
```

```
    Serial.println();
    Serial.print("MQTT connection failed: ");
    Serial.println(mqtt.connectErrorString(ret));

    mqtt.disconnect();

    delay(5000);

    Serial.print("Retrying...");
  }

  Serial.println();
  Serial.println("Adafruit IO CONNECTED!");
}

// ================================
// LOOP
// ================================

void loop()
{
  // Connect MQTT
  MQTT_connect();

  // --------------------
  // READ MQ135
  // --------------------

  int gasValue = analogRead(MQ135_PIN);

  Serial.println();
  Serial.println("==============================");
  Serial.print("MQ135 Value: ");
  Serial.println(gasValue);
```

```
// --------------------
// AIR QUALITY
// --------------------

if (gasValue < 400)
{
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);

  Serial.println("Air Quality: GOOD");

  // Publish gas value
  if (mq135Feed.publish(gasValue))
  {
    Serial.println("MQ135 data sent successfully!");
  }
  else
  {
    Serial.println("ERROR: MQ135 data NOT sent!");
  }

  // Publish air quality
  if (airQualityFeed.publish("GOOD"))
  {
    Serial.println("Air quality sent successfully!");
  }
  else
  {
    Serial.println("ERROR: Air quality NOT sent!");
  }
}
else
{
```

```
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);

    Serial.println("Air Quality: POOR");

    // Publish gas value
    if (mq135Feed.publish(gasValue))
    {
      Serial.println("MQ135 data sent successfully!");
    }
    else
    {
      Serial.println("ERROR: MQ135 data NOT sent!");
    }

    // Publish air quality
    if (airQualityFeed.publish("POOR"))
    {
      Serial.println("Air quality sent successfully!");
    }
    else
    {
      Serial.println("ERROR: Air quality NOT sent!");
    }
  }

  Serial.println("==============================");

  delay(5000);
}
```

SYSTEM WORKING

- The DHT22 measures room temperature and humidity.
- The DS18B20 sensors measure supply and return air temperatures.
- The MQ-135 monitors air-quality conditions.
- The ESP32 processes all sensor readings.
- The controller compares measurements with predefined thresholds.
- The LCD displays current HVAC parameters.
- MQTT publishes the readings to the cloud communication system.
- ThingSpeak stores the sensor data and generates graphical trends.
- The system changes its status to WARNING or CRITICAL when limits are exceeded.
- The buzzer provides a local alarm during critical conditions.
- Historical cloud data can be used for HVAC performance analysis.

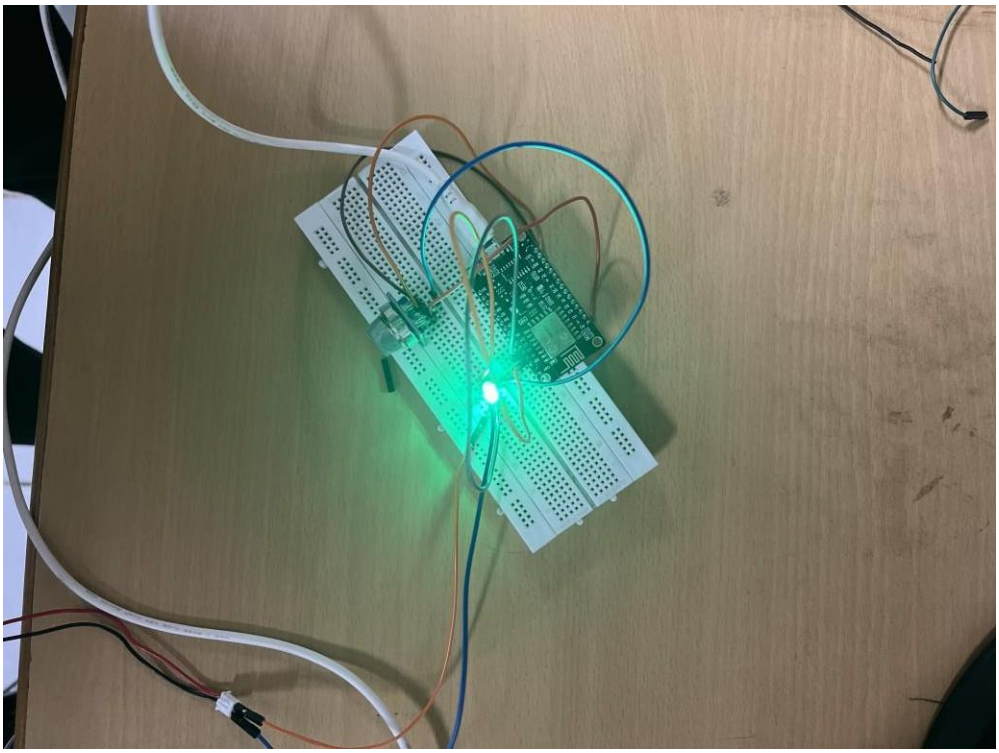
BILL OF MATERIALS

S.No	Component	Quantity	Cost (₹)
1	ESP32 Dev Board	1	600
2	DHT22 Sensor	1	250
3	DS18B20 Sensor	2	300
4	MQ-135 Air Quality Sensor	1	150
5	Relay Module	2	120
6	16×2 I2C LCD	1	180
7	Buzzer & LEDs	1 Set	80
8	Breadboard & Jumper Wires	1 Set	150
9	Power Supply	1	200
		Total	2,030

.

PROTOTYPE IMPLEMENTATION

The prototype is designed around an ESP32 controller and multiple sensors. The sensors are connected to the specified digital and analog pins. The LCD provides real-time parameter display, while the buzzer indicates critical conditions. The ESP32 communicates through Wi-Fi and MQTT, and HVAC data is stored and visualized using ThingSpeak Cloud.



RESULTS & PERFORMANCE ANALYSIS

Parameter	Normal	Warning	Critical
Room Temperature	22–25 °C	25–28 °C	>28 °C
Humidity	40–60 %	60–70 %	>70 %
Supply-Air Temperature	14–18 °C	18–20 °C	>20 °C
Return-Air Temperature	20–25 °C	25–28 °C	>28 °C
Air Quality	<100	100–150	>150

PERFORMANCE

- Real-time monitoring of HVAC environmental parameters.
- MQTT-based wireless communication for sensor data transmission.
- Remote monitoring through ThingSpeak Cloud.
- Automatic detection of abnormal temperature, humidity, and air-quality conditions.
- Local warning through buzzer during critical conditions.
- Historical data storage for HVAC performance analysis.
- Low-cost prototype suitable for educational and small-scale IoT applications.